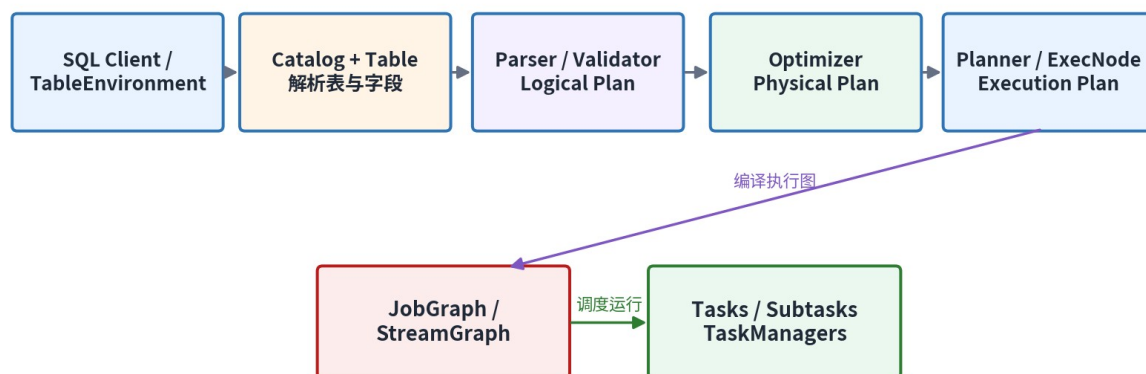


Flink 原理详解：第 11 讲

Flink SQL 执行计划与 Connector 实战：一条 SQL 最终怎样变成流任务

本讲目标：把上一讲“动态表与 Changelog”的语义，继续推进到真正的执行系统。学完后，你应该能看懂 Flink SQL 的 EXPLAIN，知道 Catalog 与 Connector 各自负责什么，并能判断一条实时 SQL 为什么需要 Exchange、State、Upsert Sink。

一条 Flink SQL 最终如何变成流式任务？



SQL 是声明式入口；真正运行的是优化器生成的物理算子和并行 Subtask。

图 1 SQL 只是入口，最终运行的是优化器生成的物理流式执行图

1. 一条 SQL 到底经历了哪些层？

例如我们写：SELECT city, SUM(amount) FROM orders GROUP BY city。人看到的是一条很短的 SQL，但 Flink 必须把它翻译成“从哪里读、按什么 Key 重分区、维护什么 State、结果产生什么 Changelog、最后写到哪里”的长期运行程序。

1. SQL Client / TableEnvironment 接收语句。
2. Catalog 解析 orders 指向哪张逻辑表，以及它的 Schema、主键、Watermark、Connector 参数。
3. Parser / Validator 检查 SQL 语法、字段、类型和函数。
4. 优化器生成并优化 Logical / Physical Plan。
5. Planner 把物理关系算子翻译为 Flink 的执行节点。

6. 执行图被调度成并行 Task/Subtask，在 TaskManager 上持续运行。

不要把 Flink SQL 想成“数据库在反复执行 SELECT”。更准确的理解是：SQL 被编译成一个长期运行的流式 Dataflow。

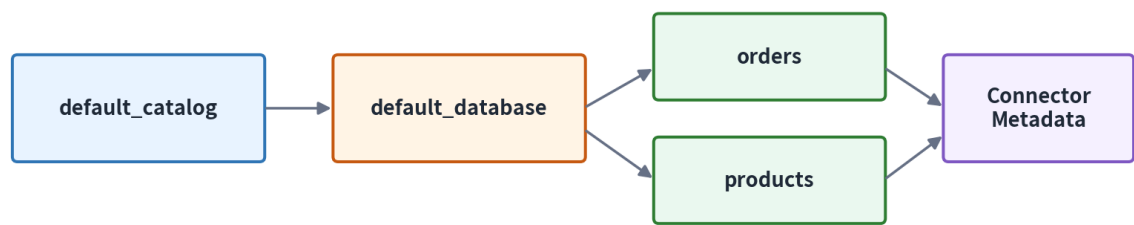
2. SQL Client、SQL Gateway、TableEnvironment 是什么关系？

入口	适合场景	你应该怎么理解
SQL Client	交互式命令行学习、调试	人在终端里直接输入 SQL
SQL Gateway	多个客户端/服务共享 SQL 会话	把 Flink SQL 暴露成服务
JDBC Driver	BI/Java 工具连接 SQL Gateway	像连数据库一样连接 Flink SQL
TableEnvironment	Java/Python 程序中的核心上下文	管理 Catalog、Table、配置并执行 SQL

无论从哪个入口进入，最终核心仍是 Table/SQL Planner + Flink Runtime。入口不同，不代表底层执行模型不同。

3. Catalog：为什么 SQL 能认识 orders？

Catalog：SQL 名字解析的“通讯录”



CREATE TABLE 注册的是 Flink 侧的表定义：字段、主键、Watermark、Connector 参数等。它告诉 Flink“怎样把外部系统解释成一张表”，不等于把外部数据复制进 Flink。

图 2 Catalog / Database / Table 构成 Flink SQL 的名字空间

Flink 的 TableEnvironment 负责连接外部系统、管理 Catalog 中的表元数据并执行 SQL。默认存在 default_catalog 与 default_database。

```

SHOW CATALOGS;
SHOW DATABASES;
SHOW TABLES;

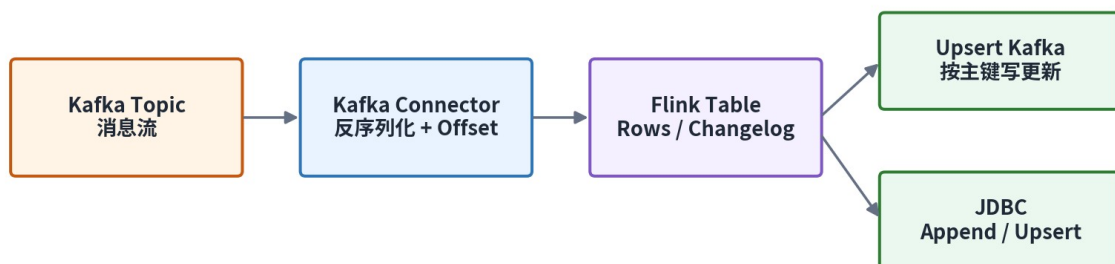
-- 查看当前 Catalog / Database 后再创建表
CREATE TABLE orders (
  order_id BIGINT,
  city STRING,
  amount DECIMAL(10,2),
  ts TIMESTAMP_LTZ(3),
  WATERMARK FOR ts AS ts - INTERVAL '5' SECOND
) WITH (...);

```

CREATE TABLE 的关键意义是“注册逻辑表定义”。数据仍然在 Kafka、文件系统、数据库等外部系统里；Connector 负责把外部数据翻译成 Flink 的 Row/Changelog。

4. Connector：Flink SQL 和现实世界之间的桥

Connector：把“外部系统”翻译成“表 / Changelog”



Connector 决定边界语义：格式、元数据、Changelog 模式、主键、Offset、Watermark、交付保证等。

图3 Connector 决定外部系统与 Flink Table/Changelog 的边界语义

Connector	典型角色	重要语义
Kafka	无界 Source / 流式 Sink	Offset、Key/Value Format、Metadata、Watermark
Upsert Kafka	Changelog Source / Upsert Sink	PRIMARY KEY 决定消息 Key；支持更新/删除
JDBC	维表 Lookup / Sink	定义主键时可消费 UPDATE/DELETE 并做 Upsert
Filesystem	文件 Source / Sink	更适合批/分区文件、Parquet 等

版本提醒：当前 Flink 2.3 官方 Kafka SQL Connector 已给出 2.3.0 依赖；但稳定文档中的 JDBC 与 Upsert Kafka 页面目前仍提示“2.3 尚无对应 connector artifact”。因此本讲把它们作为语义与架构案例；真正上机时必须按你实际 Flink 版本核对 connector JAR。

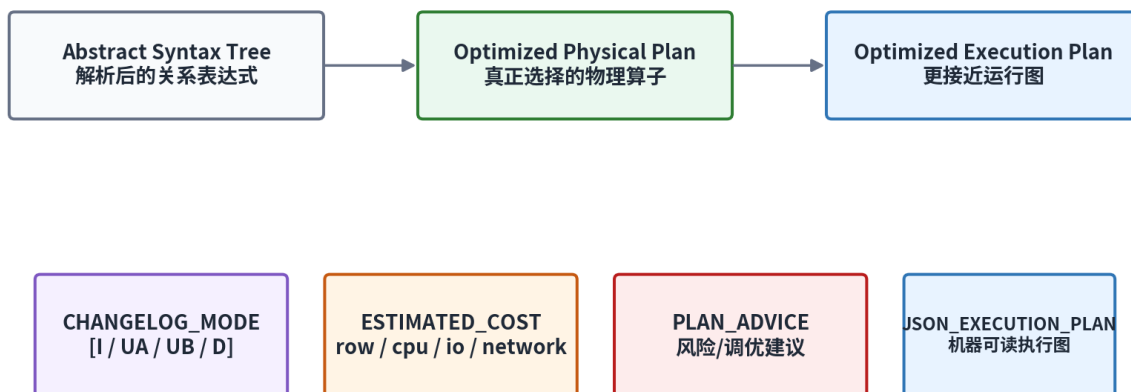
5. 一个 Kafka Source 表该怎么读？

```
CREATE TABLE orders (  
  order_id BIGINT,  
  city STRING,  
  amount DECIMAL(10,2),  
  ts TIMESTAMP_LTZ(3),  
  WATERMARK FOR ts AS ts - INTERVAL '5' SECOND  
) WITH (  
  'connector' = 'kafka',  
  'topic' = 'orders',  
  'properties.bootstrap.servers' = 'localhost:9092',  
  'properties.group.id' = 'flink-sql-demo',  
  'scan.startup.mode' = 'earliest-offset',  
  'format' = 'json'  
);
```

这段 DDL 没有把 Kafka 数据“导入” Flink。它只是告诉 Flink：orders 这张逻辑表来自哪个 Topic、怎样解析消息、从哪里开始消费，以及事件时间如何产生 Watermark。

6. EXPLAIN：学习 Flink SQL 最重要的工具之一

EXPLAIN：不要只看“SQL 长什么样”，要看“Flink 准备怎么跑”



阅读顺序：先找 Scan/Source → 再找 Exchange → 再找 Aggregate/Join → 最后看 Sink 与 Changelog。

图 4 EXPLAIN 可以从语法树一直看到更接近运行时的执行计划

```

EXPLAIN PLAN FOR
SELECT city, SUM(amount)
FROM orders
GROUP BY city;

-- Flink 2.3 还可以附加更多细节
EXPLAIN ESTIMATED_COST, CHANGELOG_MODE,
      PLAN_ADVICE, JSON_EXECUTION_PLAN
SELECT city, SUM(amount)
FROM orders
GROUP BY city;

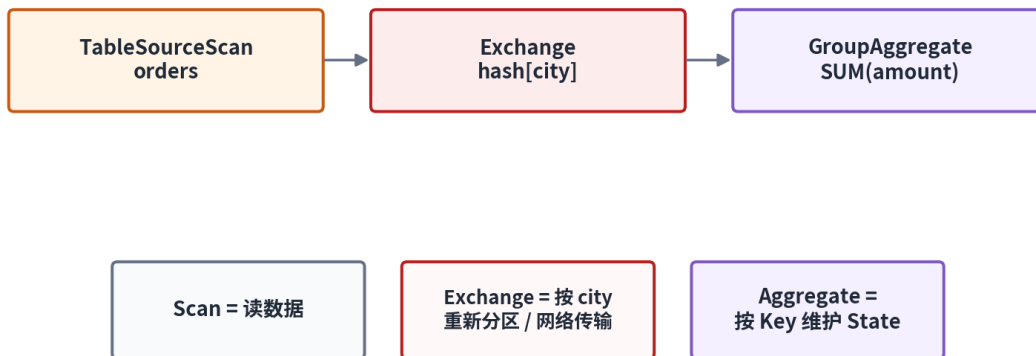
```

EXPLAIN 细节	你能看到什么
ESTIMATED_COST	优化器估算的 rows / CPU / I/O / network / memory
CHANGELOG_MODE	节点产生 I / UA / UB / D 中哪些变更
PLAN_ADVICE	潜在正确性风险与 SQL 调优建议
JSON_EXECUTION_PLAN	机器可读的 execution plan

对于零基础阶段，不需要背所有节点。先养成一个习惯：写完重要 SQL 后先 EXPLAIN，再提交执行。

7. 怎样读一个最简单的 GROUP BY 计划？

例：SELECT city, SUM(amount) FROM orders GROUP BY city



看到 **Exchange**，就要联想到：数据重分布、网络、序列化、背压和状态分区。

图 5 GROUP BY 的典型关键节点：Scan → Exchange → GroupAggregate

你可能看到类似下面的概念节点（实际文本会随版本和优化变化）：

```

GroupAggregate(groupBy=[city], select=[city, SUM(amount)])
+- Exchange(distribution=[hash[city]])
  +- TableSourceScan(table=[[..., orders]])

```

- TableSourceScan：从 Connector Source 读取外部数据。
- Exchange(hash[city])：把同一个 city 的记录送到同一个下游并行实例，本质是网络重分区。
- GroupAggregate：每个 city 维护聚合状态，并随着新事件持续更新结果。

看到 Exchange，要立即联想到“网络重分布 + 分区 + Backpressure + Checkpoint Barrier 传播”；看到 GroupAggregate，要联想到“Keyed State + Changelog”。

8. 为什么流式 GROUP BY 的 Sink 选择非常重要？

同一条动态结果，Sink 能不能正确接住？关键看 Changelog 语义



如果上游会 UPDATE/DELETE，而 Sink 只会追加，那么“语法能写”也不代表业务语义正确。

图6 上游 Changelog 模式必须与 Sink 能力匹配

城市销售额不是只输出一次。北京从 100 变成 150 时，结果表发生了 UPDATE。因此下游必须能正确表达“北京这行被更新了”。

Sink	更适合的数据语义	风险
普通 Kafka append sink	只追加的新行	不适合直接表达同一主键行的持续更新
Upsert Kafka	PRIMARY KEY 上的 INSERT/UPDATE/DELETE	可把主键映射为 Kafka key
JDBC with PRIMARY KEY	数据库行的 Upsert	需关注数据库吞吐、事务与幂等语义

“SQL 能通过语法检查”与“结果语义正确”是两回事。EXPLAIN CHANGELOG_MODE 能帮助你提前看上游到底会产生哪类变更。

9. Upsert Kafka 为什么一定要主键？

```
CREATE TABLE city_sales (  
  city STRING,  
  total_amount DECIMAL(18,2),  
  PRIMARY KEY (city) NOT ENFORCED  
) WITH (  
  'connector' = 'upsert-kafka',  
  'topic' = 'city_sales',  
  'properties.bootstrap.servers' = 'localhost:9092',  
  'key.format' = 'json',  
  'value.format' = 'json'  
);
```

Upsert Kafka 用 PRIMARY KEY 字段构造 Kafka message key。同一个 city 的后续 UPDATE 会落到相同 Key 上；DELETE 则可用 tombstone（value 为 null）表达。

当前稳定文档说明：Upsert Kafka Sink 能消费 Changelog；相同主键的消息会被分区到同一 Kafka Partition，从而保持该 Key 的更新/删除顺序。

10. JDBC Sink 为什么也要关注 PRIMARY KEY？

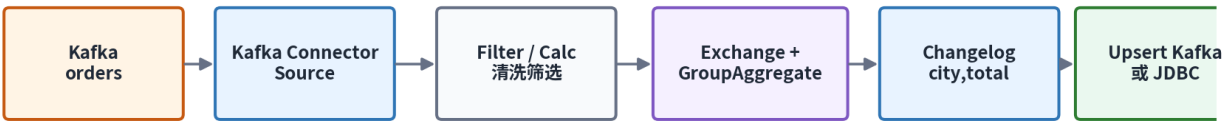
JDBC Connector 的稳定文档说明：DDL 定义主键时，JDBC Sink 可以以 Upsert 模式处理 UPDATE/DELETE；没有主键时则是 append 模式，不支持消费 UPDATE/DELETE Changelog。

```
CREATE TABLE city_sales_mysql (  
  city STRING,  
  total_amount DECIMAL(18,2),  
  PRIMARY KEY (city) NOT ENFORCED  
) WITH (  
  'connector' = 'jdbc',  
  'url' = 'jdbc:mysql://localhost:3306/demo',  
  'table-name' = 'city_sales',  
  'username' = 'root',  
  'password' = '***'  
);
```

实际部署时，JDBC Connector JAR 和数据库驱动 JAR 都必须存在，而且版本要匹配。缺依赖是 SQL Client 最常见的“表能写，运行时报找不到 Factory/Class”的原因之一。

11. 从 SQL 到完整运行图：一个城市销售额案例

完整实战链：Kafka → Flink SQL → Upsert/JDBC



EXPLAIN 让你在真正提交 Job 前，先看见 Source、Exchange、Stateful Aggregate、Sink 与 Changelog。

图 7 Kafka 订单流通过 SQL 聚合，再以 Upsert 语义写入下游

```
INSERT INTO city_sales
SELECT city, SUM(amount) AS total_amount
FROM orders
WHERE amount > 0
GROUP BY city;
```

- 1. Kafka Source Connector 读取订单并解析 JSON。
- 2. Calc/Filter 去掉 amount <= 0 的数据。
- 3. Exchange 按 city 做 Hash 重分区。
- 4. GroupAggregate 为每个 city 维护 SUM 状态。
- 5. 结果表不断发出 INSERT / UPDATE 等 Changelog。
- 6. Upsert Kafka 或 JDBC Sink 根据主键更新外部结果。

12. EXPLAIN 和 Web UI 分别回答什么？

工具	执行前/执行后	最适合回答的问题
EXPLAIN	执行前	计划里有没有 Exchange? Changelog 是什么? 用了什么 Join/Aggregate?
Web UI	执行中	哪个 Operator 忙? 谁 Backpressure? 哪个 Subtask 慢? Checkpoint 怎样?
日志	失败后	Connector/网络/序列化/权限/异常堆栈到底在哪里?

一个成熟的排查习惯是：执行前看 EXPLAIN，执行中看 Web UI，失败时再看日志。三者互补。

13. Flink SQL 常见故障应该怎么分层？

Flink SQL 出错时，先判断是哪一层

1. 语法/解析	SQL 写错、字段名不存在
2. Catalog/Schema	表未注册、类型/主键/Watermark 不对
3. Connector/JAR	缺依赖、版本不匹配、驱动缺失
4. Planner/Changelog	Sink 不接受更新流、Join/聚合语义不匹配
5. Runtime	Kafka 连不上、JDBC 超时、背压、Checkpoint 失败

先找层次，再看异常；不要看到“SQL Job 失败”就先调内存。

图 8 SQL 问题先定位层次，再动手修改

现象	优先检查
Table not found / column not found	Catalog、Database、表名、字段名
No factory found for identifier kafka	Connector JAR 是否存在，版本是否匹配
ClassNotFoundException: JDBC driver	数据库 Driver JAR
Sink doesn't support consuming update changes	上游 Changelog 与 Sink 能力是否匹配
Kafka timeout / broker unavailable	bootstrap.servers、网络、Topic、ACL
Job 能跑但越来越慢	Exchange、State、Backpressure、Sink 吞吐、Checkpoint

14. 一个可以在 SQL Client 中做的最小实验

如果暂时不想搭 Kafka/JDBC，可以先用内置 datagen + print 做“执行计划观察实验”。目标不是长期生产，而是练习 EXPLAIN 与流式 GROUP BY。

```

CREATE TABLE orders_demo (
  city STRING,
  amount INT
) WITH (
  'connector' = 'datagen',
  'rows-per-second' = '5',
  'fields.city.length' = '4',
  'fields.amount.min' = '1',
  'fields.amount.max' = '100'
);

CREATE TABLE out_demo (
  city STRING,
  total BIGINT
) WITH ('connector' = 'print');

EXPLAIN CHANGELOG_MODE
SELECT city, SUM(amount)
FROM orders_demo
GROUP BY city;

INSERT INTO out_demo
SELECT city, SUM(amount)
FROM orders_demo
GROUP BY city;

```

如果 print sink 对该结果 Changelog 的接受能力与你当前版本不匹配，先不要硬改参数；回到 EXPLAIN CHANGELOG_MODE 看结果节点到底产生什么变更。这个动作本身就是本讲最重要的实验目标。

15. 初学者最容易混淆的 8 件事

误区	正确理解
CREATE TABLE 就创建了 Kafka Topic	通常只是注册表元数据和 connector 配置
SQL 很短，所以运行一定简单	短 SQL 可能生成 Exchange、Join、Stateful Aggregate
EXPLAIN 只是打印调试信息	它是理解真实执行语义的核心工具
看到 GROUP BY 只想到求和	流式 GROUP BY 本质是长期 Keyed State
所有 Kafka Sink 都能接 UPDATE	Append Kafka 与 Upsert Kafka 的语义不同
PRIMARY KEY 会被 Flink 强制校验唯一	SQL DDL 中通常是 NOT ENFORCED，依赖数据本身满足假设
缺 Connector JAR 是 SQL 写错	这是部署/依赖层问题
Job 失败就调大内存	先区分语法、Catalog、Connector、Planner、Runtime 层

16. 练习题

1. 为什么一条 SELECT ... GROUP BY ... 在 Flink 中通常会产生 Exchange?
2. 为什么 Upsert Kafka 需要 PRIMARY KEY?
3. EXPLAIN CHANGELOG_MODE 能帮助你判断什么?
4. CREATE TABLE orders WITH (connector=kafka) 是否把 Kafka 数据存进 Flink?

5. 如果 JDBC Sink 没有主键，而上游会产生 UPDATE，会出现什么语义问题？
6. 执行前、执行中、失败后三个阶段分别应该优先看什么工具？

参考答案

1. 因为必须让相同分组 Key 的记录到同一个下游并行实例，才能维护正确的聚合状态。
2. 主键用来表示“哪一行被更新”，并映射为 Kafka key，保证同一 Key 的更新顺序。
3. 它告诉你每个物理节点可能产生 INSERT、UPDATE_BEFORE、UPDATE_AFTER、DELETE 中哪些变更。
4. 不是。它主要注册逻辑 Schema 和 Connector 参数，真正数据仍在 Kafka。
5. Append 模式无法正确消费 UPDATE/DELETE Changelog，可能无法运行或无法表达正确结果。
6. 执行前看 EXPLAIN；执行中看 Web UI；失败后看日志/异常堆栈。

17. 本最心智模型

Catalog 负责“这张表是谁”；Connector 负责“怎样和外部系统交换数据”；Planner/Optimizer 负责“SQL 应该怎样算”；EXPLAIN 负责“提前把计划给你看”；Runtime 负责“把物理算子变成持续运行的并行流任务”。

```
SQL / Table API
  ↓
Catalog 解析表与 Schema
  ↓
Connector 定义外部边界
  ↓
Logical Plan
  ↓
Optimized Physical Plan
  ↓
Exchange / Aggregate / Join / Sink
  ↓
StreamGraph / JobGraph / Tasks
  ↓
TaskManager 持续运行
  ↓
Web UI + Checkpoint + State + Backpressure
```

18. 下一讲预告

下一讲进入 Flink 第 12 讲：Flink 性能调优与生产级观察。重点会把前面分散的知识真正用于诊断：Parallelism、Task Slot、Backpressure、Checkpoint、State、Kafka Lag、SQL Exchange、Sink 吞吐，以及怎样通过 Web UI 找到真正的瓶颈。